



Faculty of Engineering and Technology

Computer Science Department

Software Engineering - COMP433

Semester 2 (Spring) 2025/2026

Phase 4: System Modelling and Design

Used-Electronics Exchange Platform

Group members:

Name	Student ID	Role	Section	Sequence diagram assigned (Task 4.4)
Anas Hamdan Al Haj	1230038	Project Manager	6	Purchase Device
Mariam Baradia	1230663	Technical Arch	6	Create Appointment
Ramz Abufarha	1230963	Requirements Engineer	6	Submit Device for Sale/Exchange
Nadeen Jaber	1220044	QA & Programmer	6	Submit Repair Request
Marah Taweel	1223094	Secretary	6	Process Refund

Prepared by:

Group No: G | Section: 6

Business No: 62

Instructor: Dr Hisham Ihshaish | Submission date: 23/6/2026

Task 4.1: Analysis Class Modelling

Lead: [Ramz Abufarha];

contributors: -

Entity classes :

#	Class Name	Description (why it exists)
E1	Customer	Registered user who browses, purchases, submits devices, books appointments, requests repairs and refunds. Primary actor in all five use cases.
E2	SalesEmployee	Employee who creates/updates device records and manages listings (Task 3.1 actor).
E3	Inspector	Employee who inspects submitted devices and assigns a condition (secondary actor of Submit Device for Sale/Exchange).
E4	Technician	Employee responsible for reviewing repair requests and updating repair status.
E5	DeliveryEmployee	Employee who updates delivery status (secondary actor of Purchase Device delivery flow).
E6	Accountant	Employee who reviews and approves refunds (secondary actor of Process Refund).
E7	Manager	Managerial user who oversees operations, accesses reports, and (per Phase 3) configures the refund policy.
E8	Device	Core item of the platform; central to purchase, submission, repair, and inspection.
E9	DeviceSubmission	Record created when a customer submits a device for sale/exchange (Submit Device use case). Holds submission type and status "Pending Inspection".
E10	Inspection	Records inspection results and condition category for a submitted device (drives sale/exchange eligibility).
E11	PurchaseTransaction	Purchase record linking customer, device, and payment (Purchase Device use case).
E12	Payment	Payment record (identifier, amount, method, date) created during purchase; referenced by refunds.
E13	Receipt	Electronic purchase receipt generated after successful payment.
E14	DeliveryRecord	Delivery record created when a customer requests delivery (Purchase A1 flow).
E15	Appointment	Inspection/exchange appointment at a branch (Create Appointment use case).
E16	Branch	Physical branch location — Ramallah and Nablus (locked decision). Appointments are booked at a branch.
E17	RepairRequest	Repair request record with status Pending/In Progress/Completed/Rejected (Submit Repair Request use case).
E18	RefundRequest	Refund record routed to the Accountant; tracks status across the refund lifecycle (Process Refund use case).
E19	RefundPolicy	Configurable policy (time window, deduction rules) the refund eligibility is validated against. Phase 3 explicitly proposes this as configurable system data.

Boundary classes

#	Class Name	Description
B1	PurchaseUI	Interface where the customer browses, selects, and purchases a device.
B2	DeviceSubmissionUI	Form for submitting a device for sale or exchange and uploading images.
B3	AppointmentUI	Booking form for selecting appointment type, branch, date, and time.
B4	RepairRequestUI	Form for submitting a repair request.
B5	RefundUI	Form for requesting a refund and attaching supporting evidence.
B6	PaymentGatewayInterface	Interface to the external Payment Gateway system (authorize, process, refund).
B7	EmailServiceInterface	Interface to the external Email Service (notifications and password recovery).

Control classes

#	Class Name	Description
C1	PurchaseController	Coordinates Purchase Device: availability check, payment, receipt generation, ownership update.
C2	SubmissionController	Coordinates Submit Device for Sale/Exchange: validation, submission record, inspection queue.
C3	AppointmentController	Coordinates Create Appointment: availability check, conflict detection, booking.
C4	RepairController	Coordinates Submit Repair Request: validation, duplicate check, record creation.
C5	RefundController	Coordinates Process Refund: eligibility validation, accountant routing, gateway refund.

Relationships:

The relationships below were derived directly from the Phase 3 use-case specifications.

Composition is used only where a part cannot exist independently of its whole (a Payment and Receipt belong to a PurchaseTransaction; an Inspection belongs to a DeviceSubmission).

Dependencies («use») are used for control-to-external-system links and for validation against the refund policy.

Source	Relationship	Target	Multiplicity	Reason
Customer	association	PurchaseTransaction	1 → 0..*	A customer makes many purchases.
Customer	association	DeviceSubmission	1 → 0..*	A customer submits many devices.
Customer	association	Appointment	1 → 0..*	A customer books many appointments.
Customer	association	RepairRequest	1 → 0..*	A customer files many repair requests.
Customer	association	RefundRequest	1 → 0..*	A customer requests refunds for eligible transactions.
PurchaseTransaction	association	Device	0..* → 1	Each purchase is for exactly one device.
PurchaseTransaction	composition	Payment	1 → 1	A purchase owns its payment record.
PurchaseTransaction	composition	Receipt	1 → 1	The receipt belongs to its purchase.
PurchaseTransaction	association	DeliveryRecord	1 → 0..1	A delivery record exists only if delivery was requested.
DeliveryRecord	association	DeliveryEmployee	0..* → 0..1	A delivery employee updates the delivery status.
DeviceSubmission	association	Device	1 → 1	A submission concerns one device.
DeviceSubmission	composition	Inspection	1 → 0..1	An inspection belongs to its submission.
Inspection	association	Inspector	0..* → 1	An inspector records inspection results.
Appointment	association	Branch	0..* → 1	Each appointment is at one branch.
RepairRequest	association	Device	0..* → 1	Each repair request targets one device.
RepairRequest	association	Technician	0..* → 0..1	A technician reviews the repair request.
RefundRequest	association	PurchaseTransaction	0..* → 0..1	A refund references the original transaction.
RefundRequest	association	Accountant	0..* → 0..1	The accountant reviews the refund.
RefundRequest	dependency	RefundPolicy	«use»	Eligibility is validated against the policy.
RefundPolicy	association	Manager	0..* → 1	The manager configures the refund policy.
Device	association	SalesEmployee	0..* → 1..1	A sales employee maintains device records.

Source	Relationship	Target	Multiplicity	Reason
Each UI boundary	association	its Controller	1 → 1	The boundary forwards user actions to its control class.
PurchaseController / RefundController	dependency	PaymentGatewayInterface	«use»	Purchase and refund send requests to the gateway.
PurchaseController / RefundController	dependency	EmailServiceInterface	«use»	Both send confirmation notifications.

Final UML Analysis Class Diagram

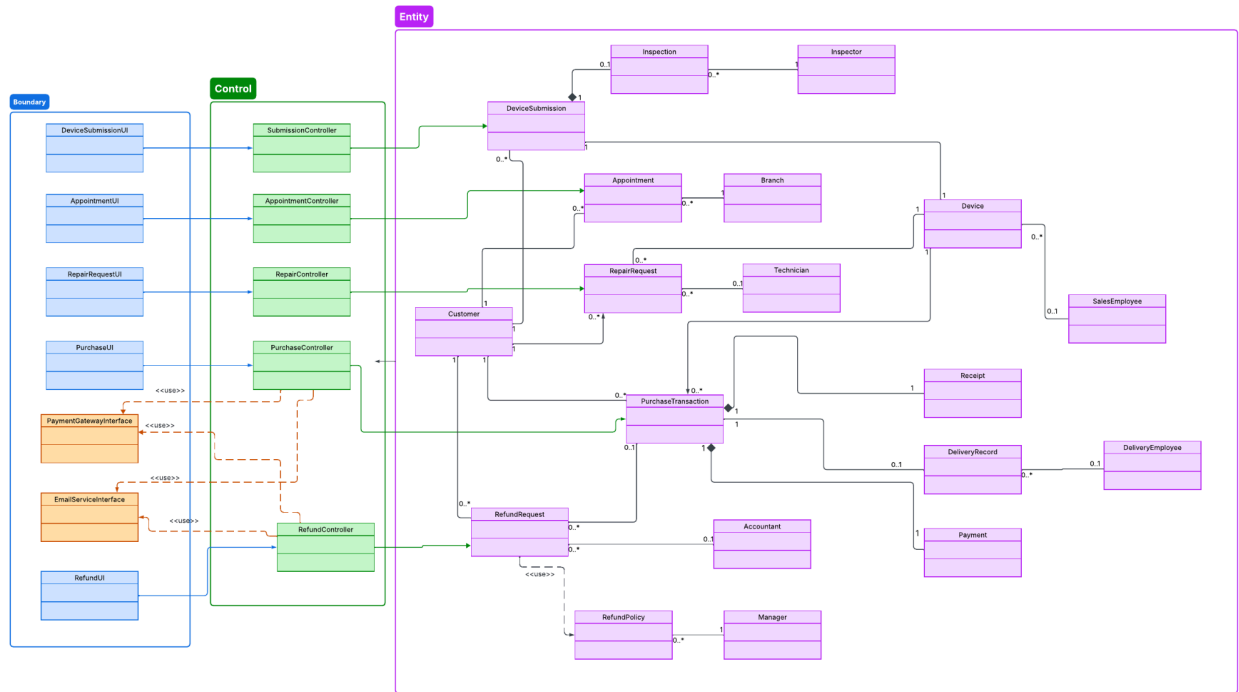
The diagram is laid out in the standard analysis arrangement — boundary classes on the left, control classes in the centre, and entity classes on the right. Boundary classes are blue, control classes green, entity classes purple, and the two external systems amber. Associations carry multiplicities at both ends, compositions are shown with a filled diamond at the owning class, and «use» dependencies are dashed.

Phase 3 Use Case	Boundary	Control	Entity classes involved
Purchase Device	PurchaseUI	PurchaseController	Customer, Device, PurchaseTransaction, Payment, Receipt, DeliveryRecord, DeliveryEmployee, PaymentGateway, EmailService
Submit Device for Sale/Exchange	DeviceSubmissionUI	SubmissionController	Customer, Device, DeviceSubmission, Inspection, Inspector
Create Appointment	AppointmentUI	AppointmentController	Customer, Appointment, Branch
Submit Repair Request	RepairRequestUI	RepairController	Customer, RepairRequest, Device, Technician
Process Refund	RefundUI	RefundController	Customer, RefundRequest, PurchaseTransaction, RefundPolicy, Accountant, PaymentGateway, EmailService

All five key use cases and all nine Phase 3 actors (Customer, Sales Employee, Inspector, Technician, Delivery Employee, Accountant, Manager, Payment Gateway, Email Service) are represented in the model.

[Analysis_class_diagram - pdf](#)

[analysis_class_diagram - lucid.app](#)



Task 4.2: Detailed Class Modelling

Lead: [Mariam Baradia];

Contributors: [Ramz Abufarha: Reviewed the detailed class model and implementation consistency.]

The detailed class diagram refines the Analysis Class Diagram (Task 4.1) into an implementation-ready model, keeping the same thirty-one classes under the Boundary–Control–Entity pattern (boundary left, control centre, entities right). No classes were added or removed; each was enriched with attributes and methods from the Phase 2 requirements and Phase 3 use cases — boundary classes with display operations, control classes with coordination operations, and entities with their state (e.g. Device with category, brand, status, `updateStatus()`, `isAvailable()`), using '-' for private attributes and '+' for public operations. All analysis relationships are retained with multiplicities, including Customer 1 → 0..* PurchaseTransaction, DeviceSubmission, Appointment, RepairRequest, and RefundRequest; the compositions PurchaseTransaction ♦ Payment/Receipt and DeviceSubmission ♦ Inspection; and «use» dependencies to PaymentGatewayInterface, EmailServiceInterface, and RefundPolicy.

This structure gives high cohesion (each class owns one responsibility), low coupling (BCE separation confines external access to the interface classes), maintainability (rules isolated in classes like RefundPolicy), and scalability (new use cases add as a boundary–control pair without disturbing existing classes).

[Detailed Class Modelling](#)

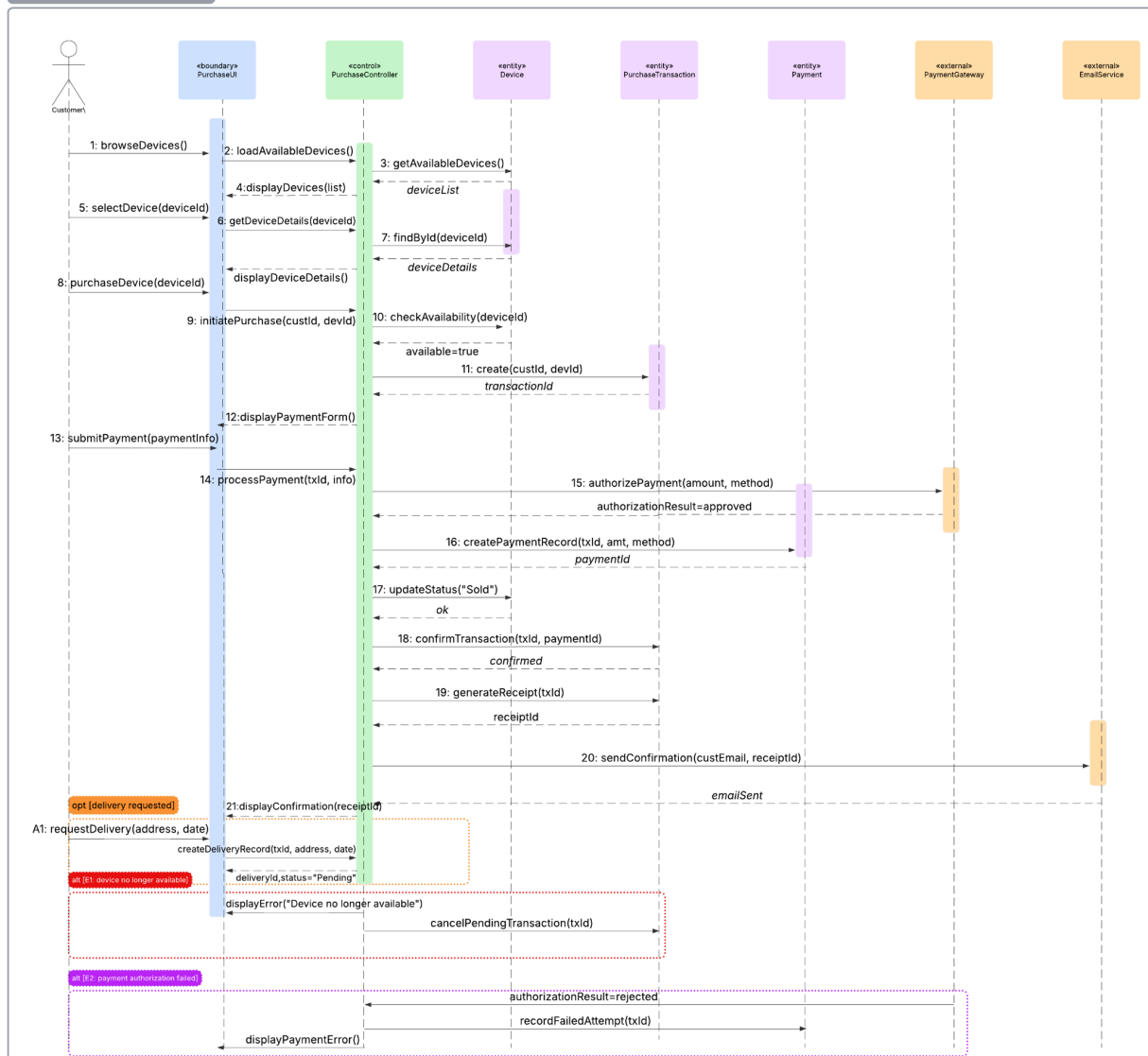
Task 4.3: Sequence Modelling

Purchase Device Sequence Diagram

Author: Anas Hamdan Al Haj

This sequence diagram models the **Purchase Device** use case, showing the interaction between the Customer, PurchaseUI, PurchaseController, Device, PurchaseTransaction, Payment, PaymentGateway, and EmailService. The main flow covers device browsing, availability verification, payment processing, receipt generation, and email confirmation. It also includes an opt fragment for optional delivery (A1) and two alt fragments for exceptions E1 (device unavailable) and E2 (payment failure), mapping to UR2, SR2.1–SR2.6, SR9.1, and SR11.1–SR11.6.

Sequence diagram



Purchase Device Sequence Diagram

Submit Device for Sale/Exchange Sequence Diagram

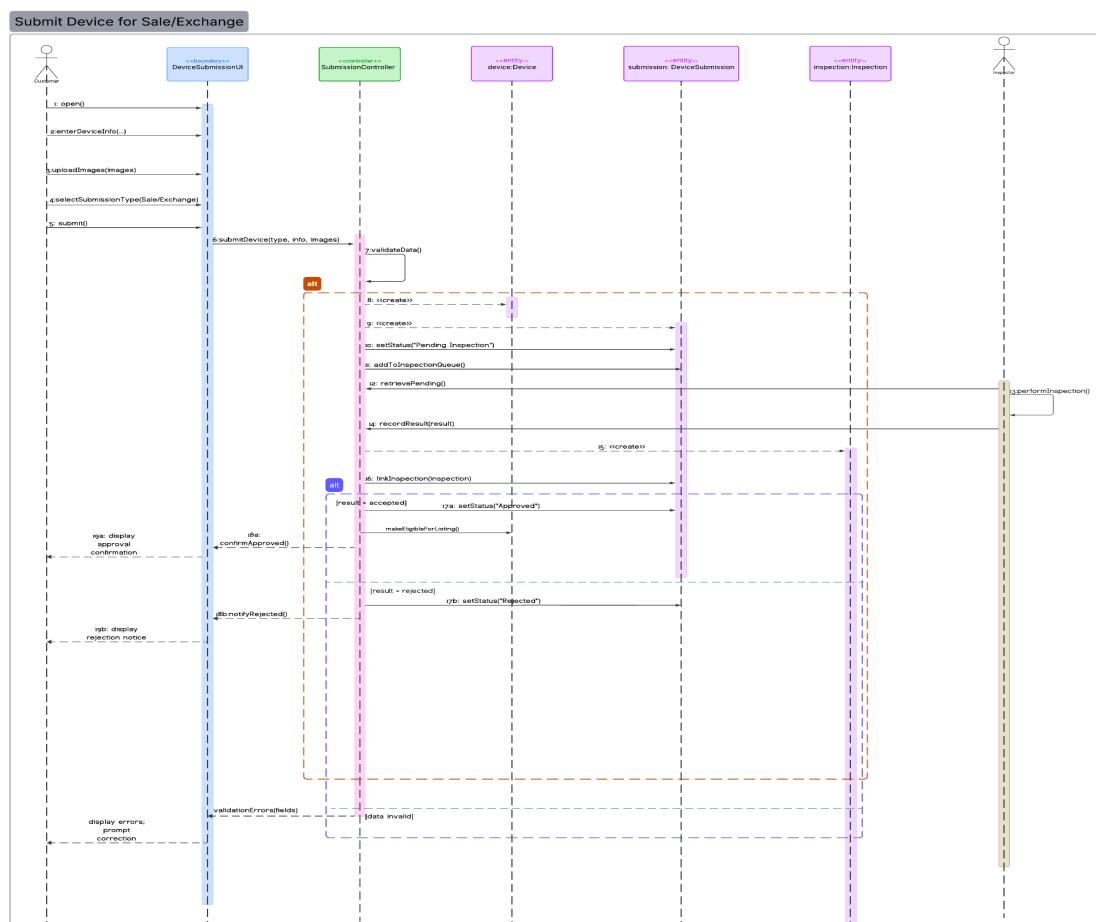
Author: Ramz Abufarha

This sequence diagram models the dynamic behaviour of the Submit Device for Sale/Exchange use case, following the Boundary-Control-Entity pattern from the Analysis Class Diagram. The Customer interacts only with the DeviceSubmissionUI, which forwards the request to the SubmissionController; the controller validates the data, creates the Device and DeviceSubmission entities, sets the status to "Pending Inspection", and queues the submission for the Inspector, who inspects the device and records a result that creates a linked Inspection entity.

Two alternative paths are shown: invalid data returns errors for correction with no record created, while the inspection outcome sets the submission to "Rejected" (Customer notified) or "Approved" (eligible for listing). Expressing validation and inspection outcomes as alternative fragments lets the diagram capture the ordered message flow and conditional logic of the use case at runtime, using names consistent with the Analysis Class Diagram.

[Submit Device for Sale/Exchange - Sequence diagram](#)

[Submit Device for Sale/Exchange - PDF](#)



Process Refund Sequence Diagram

Author: Marah Taweel.

This sequence diagram models the dynamic behaviour of the Process Refund use case, following the Boundary-Control-Entity pattern from the Analysis Class Diagram. The Customer interacts only with the RefundUI, which forwards requests to the RefundController; the controller validates the request against the RefundPolicy, manages the refund lifecycle, and coordinates with the Accountant, PaymentGateway, and EmailService to complete or reject the refund.

The main flow covers refund form display, eligibility validation (refund age and condition checks), refund record creation with status "Pending Review", and Accountant notification. Two alt fragments are used: the first captures the eligibility outcome — either creating a RefundRequest and routing it to the Accountant, or immediately rejecting it and notifying the Customer; the second captures the Accountant's decision — either processing the approved amount through the PaymentGateway, updating statuses to "Completed" and "Refunded", and sending an email confirmation, or rejecting the request and notifying the Customer with a reason. An opt fragment covers Alternative Flow A1 (partial refund), and a final alt fragment covers Exception Flow E3 (payment gateway failure), where the request is flagged for manual processing and the Customer is notified of the delay.

The diagram maps to UR12 and the detailed requirements SR12.1–SR12.4 from Phase 2, using class names consistent with the Analysis Class Diagram.

[Process Refund Sequence Diagram](#)

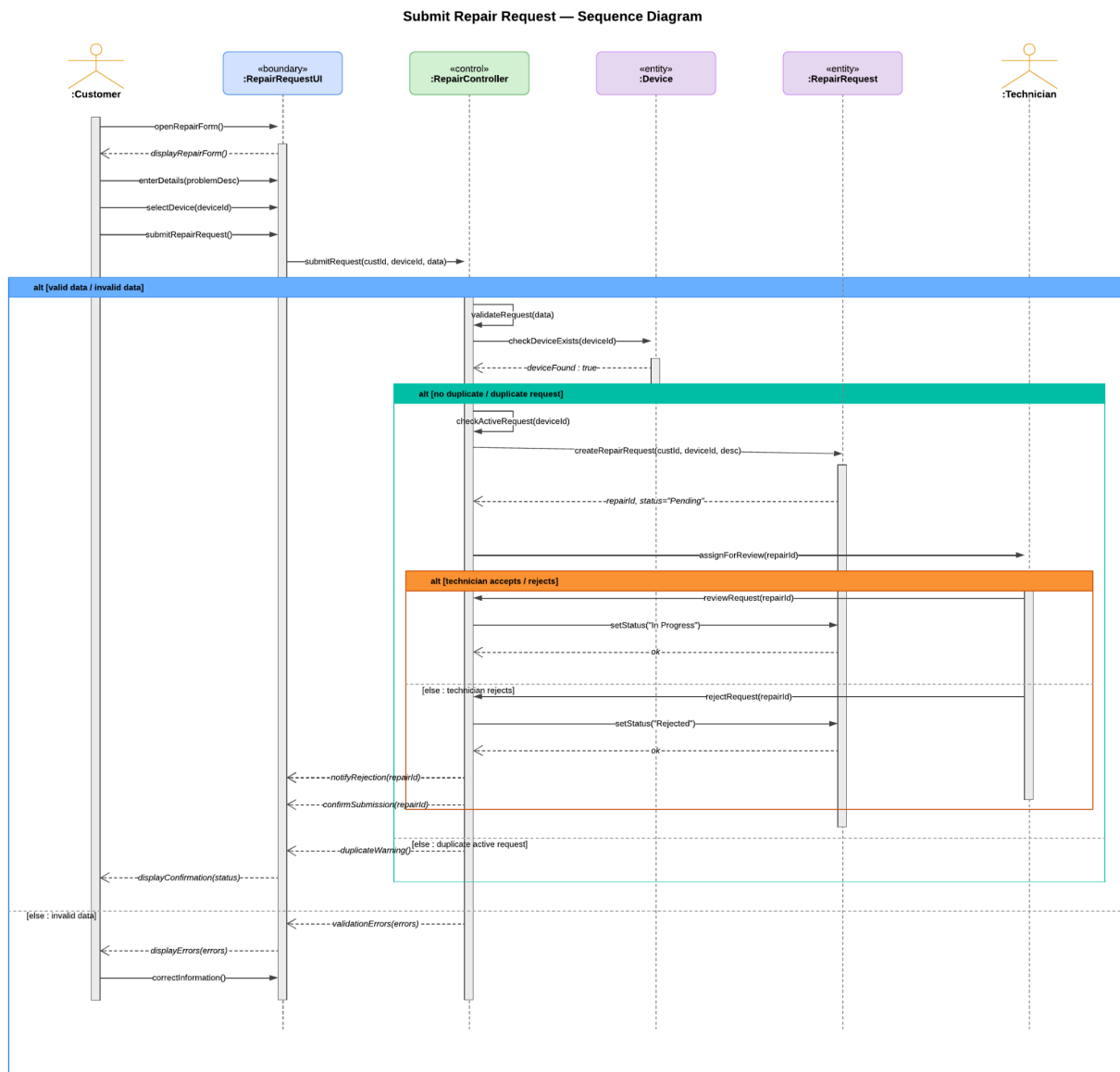
[illegible]

Submit Repair Request Sequence Diagram

Author: Nadeen Jaber

This sequence diagram models the dynamic behaviour of the Submit Repair Request use case, following the Boundary-Control-Entity pattern from the Analysis Class Diagram. The Customer interacts only with the RepairRequestUI, which forwards the request to the RepairController; the controller validates the submitted data, checks whether the selected device already has an active repair request, creates the RepairRequest entity with status set to "Pending", stores it, and assigns it to the Technician queue for review.

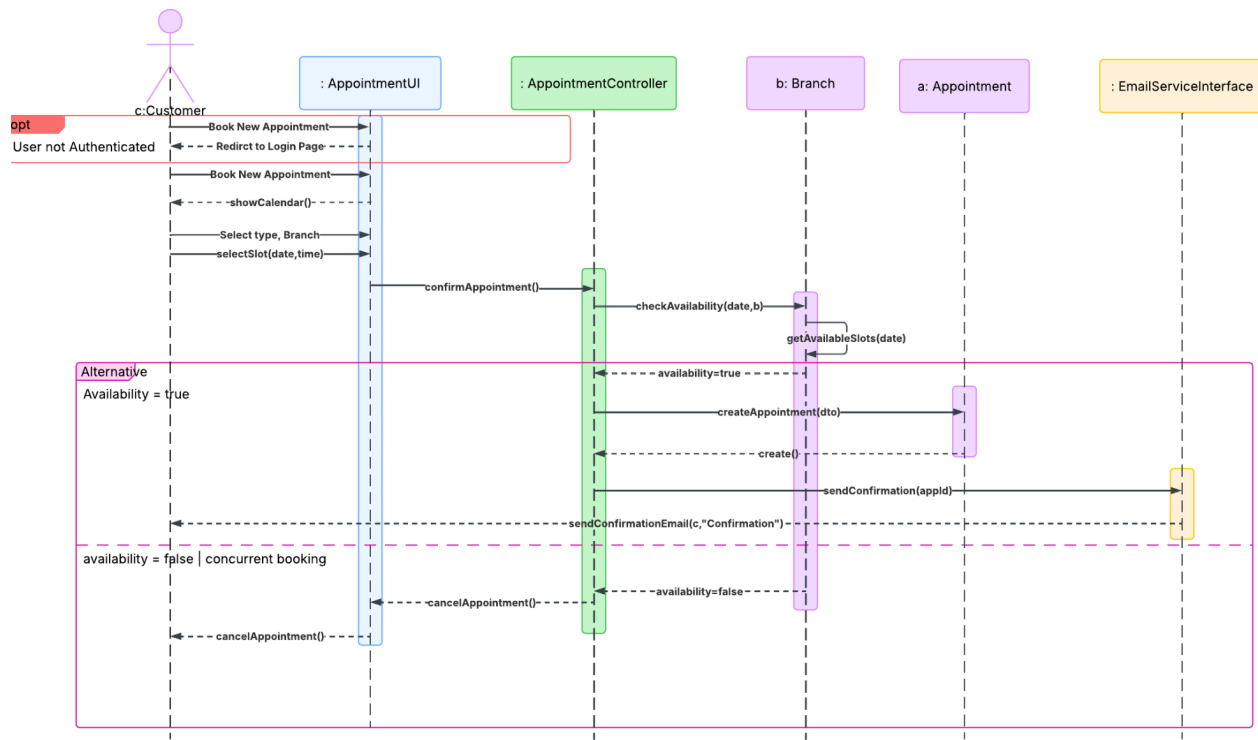
Three alternative paths are shown: invalid or incomplete data returns validation errors to the Customer with no record created; a duplicate active repair request triggers a warning and prevents a new record from being created; and the Technician review outcome either sets the status to "In Progress" (accepted) or "Rejected" (with the Customer notified accordingly). Expressing validation, duplicate detection, and review outcomes as nested alternative fragments lets the diagram capture the ordered message flow and conditional logic of the use case at runtime, using class and method names consistent with the Analysis Class Diagram.



[Submit Repair Request Sequence Diagram](#)

Create Appointment Sequence Diagram

Author: Mariam Baradeiya



[Create Appointment Sequence Diagram Link](#)

Task 4.4: Component Design

Lead: Anas Hamdan Al Haj

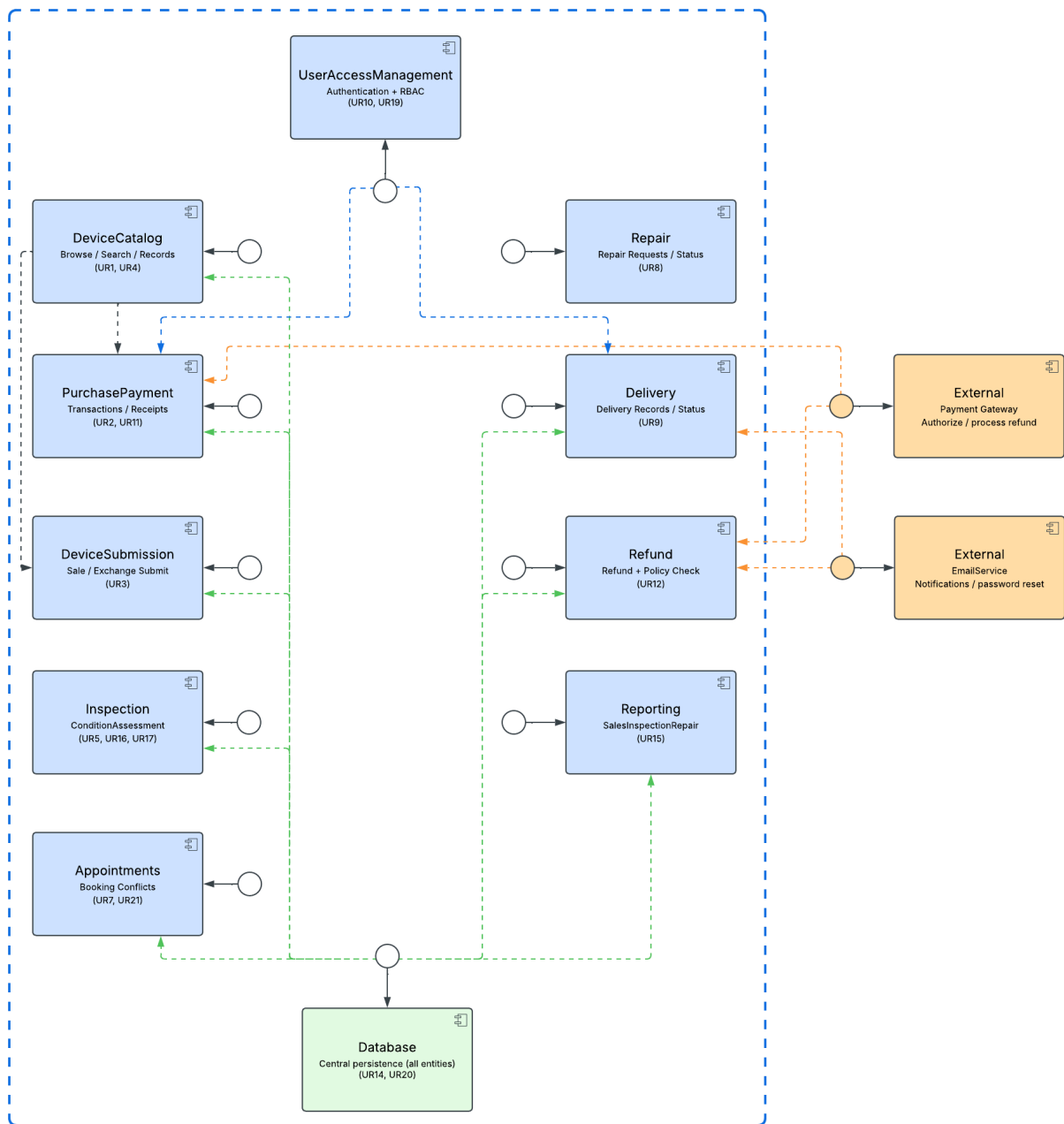
The component diagram decomposes the Used-Electronics Exchange Platform into ten internal components, each grouping a cohesive set of classes around a single business capability, plus two external systems. Each component exposes its functionality through **provided interfaces** (lollipops) and consumes services from others through **required interfaces** (sockets), so components communicate only through well-defined contracts rather than direct dependencies — supporting the design goals of high cohesion and low coupling.

The **UserAccessManagement** component sits at the centre of the design, providing `IAuthentication` and `IAccessControl`. Every functional component depends on it to enforce role-based access before any

operation is performed, which realises the role-based security design goal (UR19). The **Database** component provides a single IDatabase interface that all components rely on for persistence (UR14, UR20), keeping data access centralised in one tier.

The functional components map directly to the platform's user requirements: **DeviceCatalog** handles browsing and device records (UR1, UR4); **PurchasePayment** manages transactions and receipts (UR2, UR11); **DeviceSubmission** handles sale/exchange submissions (UR3); **Inspection** performs condition assessment (UR5, UR16, UR17); **Appointment** manages booking and conflict detection (UR7, UR21); **Repair** handles repair requests and status tracking (UR8); **Refund** processes refunds against the refund policy (UR12); and **Reporting** generates business reports (UR15).

The two external systems lie outside the system boundary. **PaymentGateway** provides IPaymentGateway, used by PurchasePayment and Refund to authorise payments and process refunds. **EmailService** provides INotification, used by the components that send confirmations and status updates to customers. Dependency lines are colour-coded by the type of interface required (database access, access control, or external system) so the flow of dependencies across the system remains easy to trace.



Component Diagram

Task 4.5: Deployment Design

Lead: Nadeen Jaber

The deployment diagram maps the platform's software components onto physical hardware nodes, organised into the four tiers shown in the legend: the Client Tier, Application Tier, Data Tier, and external services. Each node is drawn as a 3D cube representing a separate execution environment, and the communication paths between nodes are labelled with the protocol or API used.

The **Customer Device** node (Client Tier) hosts the user-facing components — the Web Browser and Mobile App — through which customers and employees interact with the system. This node communicates with the Application Server over **HTTPS**, ensuring all client–server traffic is encrypted.

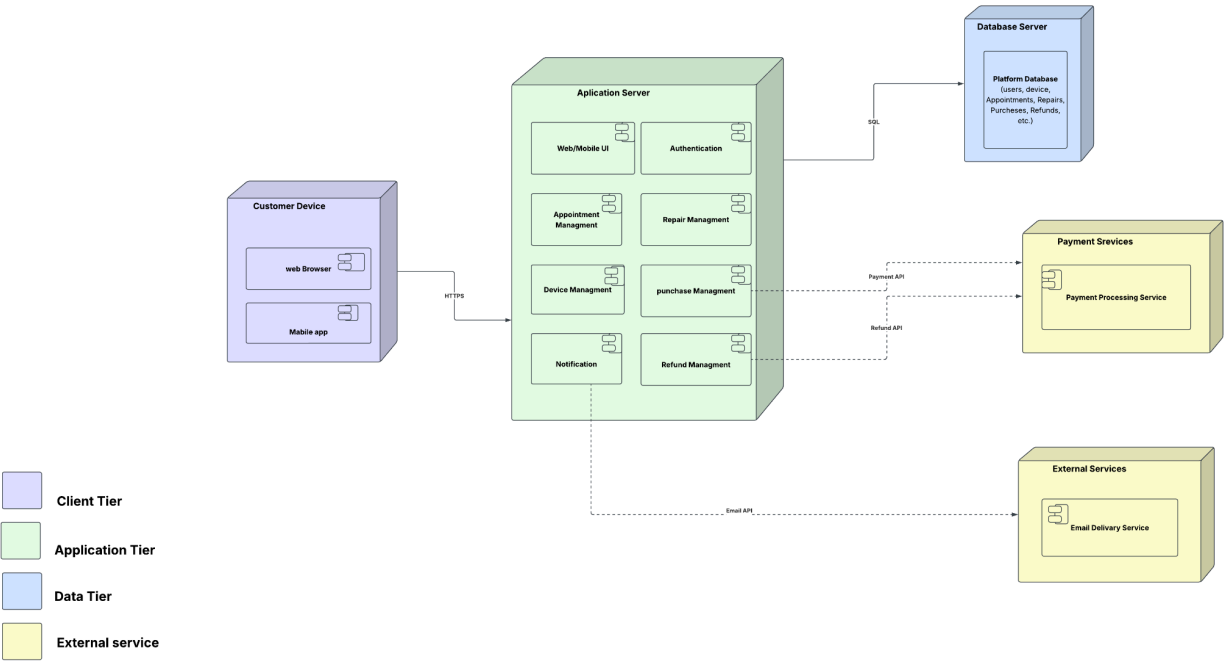
The **Application Server** node (Application Tier) is the core of the deployment and hosts all business-logic components: Web/Mobile UI, Authentication, Appointment Management, Repair Management, Device Management, Purchase Management, Notification, and Refund Management. Centralising these components on one server keeps the business logic cohesive while isolating it from both the presentation and data tiers.

The **Database Server** node (Data Tier) hosts the Platform Database, which stores all persistent records — users, devices, appointments, repairs, purchases, and refunds. The Application Server communicates with it over a dedicated **SQL** connection, keeping data access confined to a single node for security and maintainability.

Two external service nodes lie outside the organisation's infrastructure. The **Payment Services** node hosts the Payment Processing Service, reached by the Purchase Management and Refund Management components through the **Payment API** and **Refund API** respectively. The **External Services** node hosts the Email Delivery Service, reached by the Notification component through the **Email API**, and is used to send confirmations and status updates to customers.

This separation of nodes reflects the three-tier layered architecture and supports the design goals: low coupling between independently deployable tiers, high cohesion within each node, and centralised, controlled access to the data and authentication layers in line with the role-based security goal (UR19).

Deployment Diagram - Used-Electronic Exchange Platform



[Deployment Diagram](#)